

PERSISTENCIA EN CASCADA



Guía: Persistencia en Cascada con JPA y Hibernate



Objetivo

Aprender a guardar un objeto principal y sus objetos relacionados automáticamente en la base de datos usando **persistencia en cascada** con Hibernate y JPA.

1. Modificamos la clase `Alumno.java`

Abrimos el archivo:

```
src/main/java/sga/domain/Alumno.java
```

Localizamos las relaciones `@OneToOne` con las entidades `Domicilio` y `Contacto`, y les agregamos la configuración de **persistencia en cascada** usando:

```
@OneToOne(cascade = CascadeType.ALL)
```

```

@JoinColumn(name = "id_domicilio", referencedColumnName = "id_domicilio")
private Domicilio domicilio;

@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "id_contacto", referencedColumnName = "id_contacto")
private Contacto contacto;

```

✅ **Importante:** Esto indica que cualquier operación realizada sobre `Alumno` se propagará automáticamente a `Domicilio` y `Contacto` (INSERT, UPDATE, DELETE, etc.).

💡 Código completo `Alumno.java`:

```

package sga.domain;

import java.io.Serializable;
import java.util.List;
import java.util.Objects;
import jakarta.persistence.*;

@Entity
public class Alumno implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_alumno")
    private Integer idAlumno;

    private String nombre;

    private String apellido;

    @JoinColumn(name="id_domicilio", referencedColumnName = "id_domicilio")
    @ManyToOne(cascade = CascadeType.ALL)
    private Domicilio domicilio;

    @JoinColumn(name="id_contacto", referencedColumnName = "id_contacto")
    @ManyToOne(cascade = CascadeType.ALL)
    private Contacto contacto;

    @OneToMany(mappedBy = "alumno")
    private List<Asignacion> asignaciones;

    public Alumno() {
    }
}

```

```
public Alumno(Integer idAlumno) {
    this.idAlumno = idAlumno;
}

public Integer getIdAlumno() {
    return idAlumno;
}

public void setIdAlumno(Integer idAlumno) {
    this.idAlumno = idAlumno;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public Domicilio getDomicilio() {
    return domicilio;
}

public void setDomicilio(Domicilio domicilio) {
    this.domicilio = domicilio;
}

public Contacto getContacto() {
    return contacto;
}

public void setContacto(Contacto contacto) {
    this.contacto = contacto;
}
```

```
public List<Asignacion> getAsignaciones() {
    return asignaciones;
}

public void setAsignaciones(List<Asignacion> asignaciones) {
    this.asignaciones = asignaciones;
}

@Override
public String toString() {
    return "Alumno{" + "idAlumno=" + idAlumno + ", nombre=" + nombre + ", apellido=" + apellido + ",
domicilio=" + domicilio + ", contacto=" + contacto + '}';
}

@Override
public int hashCode() {
    int hash = 5;
    hash = 29 * hash + Objects.hashCode(this.idAlumno);
    return hash;
}

@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    final Alumno other = (Alumno) obj;
    if (!Objects.equals(this.idAlumno, other.idAlumno)) {
        return false;
    }
    return true;
}
}
```

2. Creamos la clase de prueba `PersistenciaCascadaTest.java`

Ubicación:

src/main/java/sga/test/cascade/PersistenciaCascadaTest.java

💡 Código completo **PersistenciaCascadaTest.java**:

```
package sga.test.cascade;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import sga.domain.*;

public class PersistenciaCascadaTest {

    public static void main(String[] args) {
        // Crear EntityManager y Factory con try-with-resources
        try (
            EntityManagerFactory emf = Persistence.createEntityManagerFactory("HibernateJpaPU");
            EntityManager em = emf.createEntityManager()
        ) {
            // Crear objetos dependientes
            Domicilio domicilio = new Domicilio();
            domicilio.setCalle("Nogales");
            domicilio.setNoCalle("10");
            domicilio.setPais("México");

            Contacto contacto = new Contacto();
            contacto.setEmail("clara@mail.com");
            contacto.setTelefono("44332211");

            // Crear entidad principal
            Alumno alumno = new Alumno();
            alumno.setNombre("Carlos");
            alumno.setApellido("Lara");
            alumno.setDomicilio(domicilio); // Relación uno a uno con cascade
            alumno.setContacto(contacto);   // Relación uno a uno con cascade

            // Persistir con manejo adecuado de la transacción
            try {
                em.getTransaction().begin();
                em.persist(alumno); // Hibernate se encarga de persistir en cascada
                em.getTransaction().commit();
                System.out.println("Alumno persistido correctamente: " + alumno);
            } catch (Exception e) {
                if (em.getTransaction().isActive()) {
                    em.getTransaction().rollback();
                }
            }
        }
    }
}
```

```
    }  
    System.err.println("Error al persistir el alumno: " + e.getMessage());  
}  
  
} catch (Exception ex) {  
    System.err.println("Error al inicializar EntityManager: " + ex.getMessage());  
}  
}  
}
```

3. Ejecutamos la clase

- Hacemos clic derecho sobre `PersistenciaCascadaTest.java`
- Seleccionamos **Run File**
- Observamos en consola que:
 - Se hace `INSERT` en `Contacto`
 - Luego `INSERT` en `Domicilio`
 - Y finalmente `INSERT` en `Alumno`



Hibernate resuelve automáticamente el orden correcto para respetar las dependencias.

4. Verificamos en la base de datos

Revisamos las tres tablas (`alumno`, `domicilio`, `contacto`) y confirmamos que:

- Se insertaron correctamente los tres registros.
 - Se generaron los IDs automáticamente.
 - La tabla `alumno` guarda los ID de `domicilio` y `contacto` como llaves foráneas.
-



Conclusión

Gracias a la configuración `cascade = CascadeType.ALL`, es posible guardar un objeto principal (como `Alumno`) junto con sus relaciones (`Domicilio`, `Contacto`) **en una sola transacción** y con **una sola línea de persistencia**.

Esto simplifica mucho el código y garantiza la integridad de los datos.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)